AllMinds

Frontend, Integrations & APIs

How the Next.js teacher dashboard is built, every API it calls, and how it reaches the backend and the AI agents

Neurodivergent-affirming AI teaching companion — HackAstone Agentic AI

Generated from the source of the HackAstone-Agentic-AI/AllMinds repository

1. What AllMinds is trying to achieve

AllMinds is a teacher-facing platform for South African educators who teach neurodivergent children aged 3-10 (autism, ADHD, dyslexia, dyscalculia, sensory processing differences, PDA and twice-exceptional profiles). **The primary goal** is to give a non-specialist teacher, in seconds, classroom-ready support that would otherwise need a specialist: DCAPS-aligned lesson plans and worksheets, visual supports, parent letters, learner progress analytics, schedules and a 24/7 AI colleague ("Thandi") - in any of South Africa's 11 official languages.

The system is a three-service monorepo: a **Next.js frontend** (apps/frontend), an **Express + PostgreSQL backend** (apps/backend) and a **FastAPI multi-agent service** (apps/agents). This document covers the frontend and every API it touches. PDF 2 covers the backend/database and PDF 3 the agents.

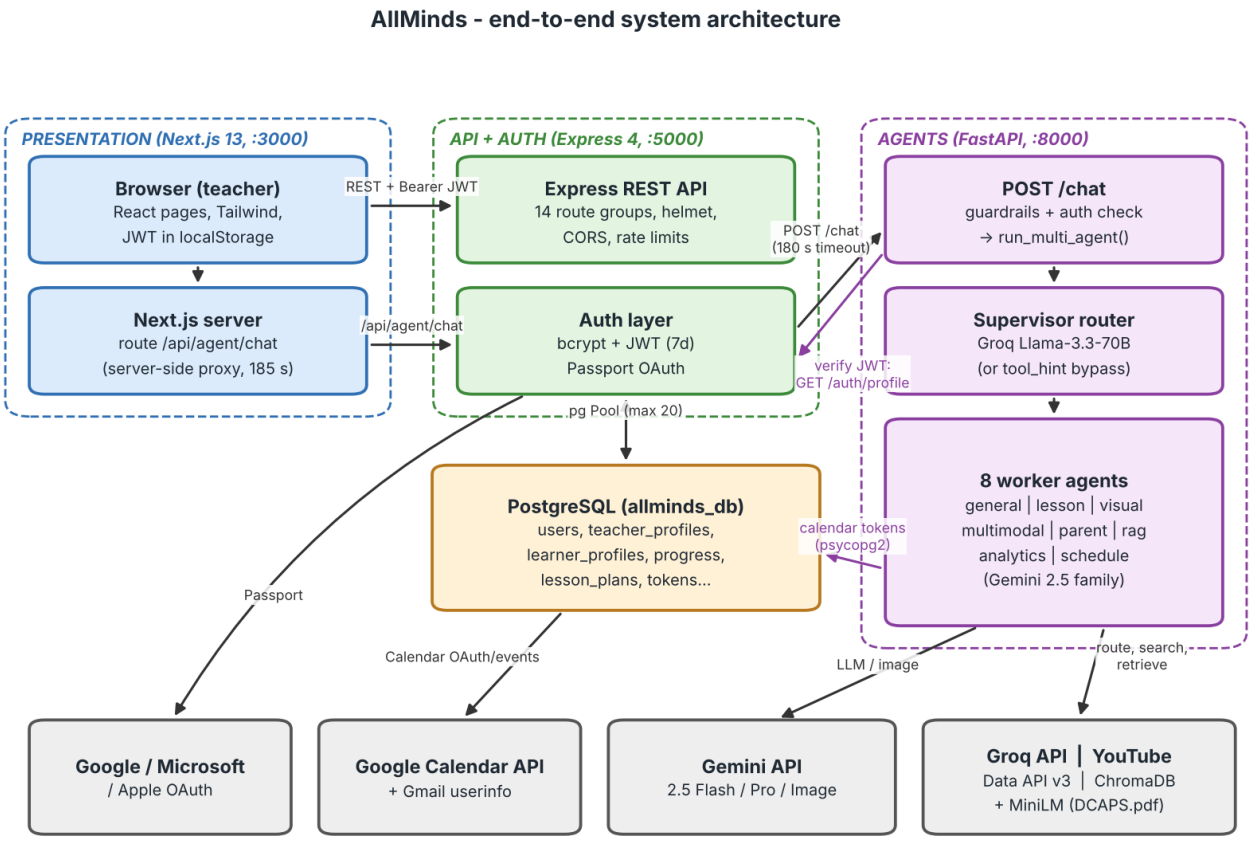


Figure 1. The whole system. The browser only ever talks to Next.js and Express; it never calls the agents or the LLM providers directly.

1.1 Technology stack (apps/frontend/package.json)

Concern	Choice	Why it matters
Framework	Next.js 13.5 App Router, React 18.2, TypeScript 5.2	File-based routing under app/; server route handlers (app/api/...) let us proxy AI calls server-side.
Styling	Tailwind CSS 3.3 + globals.css	Utility classes; theme (light/dark/auto) applied by PreferencesContext.
Icons	lucide-react	Consistent icon set across dashboard and tools.
Exports	jsPDF 4.2 (lesson plans, parent letters), ExcelJS 4.4 (learner lists)	Teachers take artefacts offline / print them.
State	React hooks + 2 contexts (LanguageContext, PreferencesContext)	No Redux; server is the source of truth, local state is per page.
HTTP client	lib/api.ts - one fetch wrapper	Adds JWT, JSON headers, unified ApiError, 401 -> session-expired event.

Concern	Choice	Why it matters
Hosting	Render (render.yaml) or Netlify (netlify.toml + @netlify/plugin-nextjs)	NEXT_PUBLIC_API_URL / BACKEND_API_URL point at the Express service.

Table 1. Frontend stack

2. Application structure

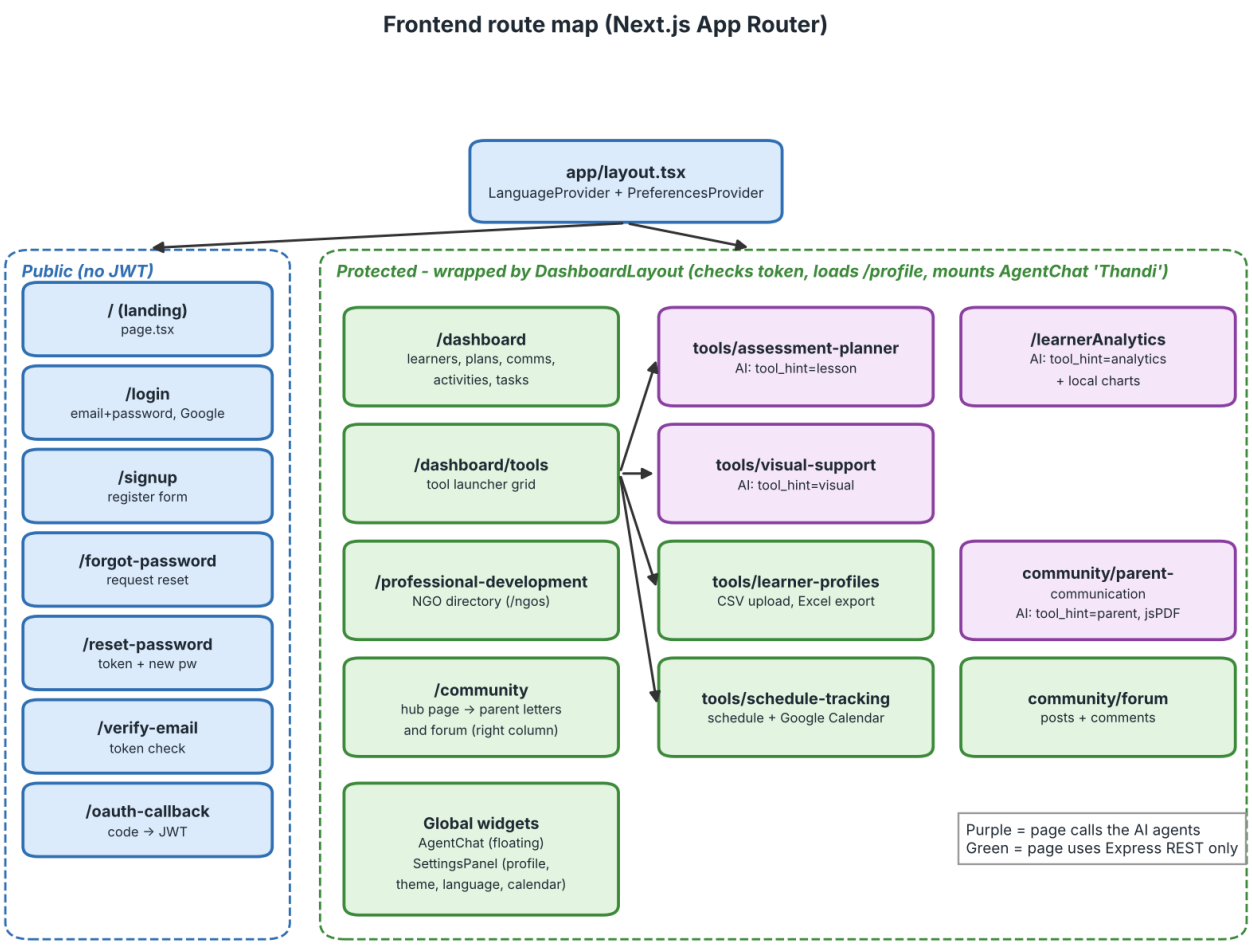


Figure 2. Every route in apps/frontend/app. Purple pages call the AI agents through api.chatWithAgent(); green pages only use the Express REST API.

2.1 Shell and shared components

- **app/layout.tsx** - root layout; wraps everything in *LanguageProvider* (UI translations from lib/i18n/translations.ts, code stored in localStorage 'allminds_language') and *PreferencesProvider* (theme, font size and accessibility preferences, persisted in localStorage and synced from the server profile's theme_preference).
- **components/DashboardLayout.tsx** - the auth guard for every /dashboard page. On mount it reads 'allminds_token'; if missing it redirects to /login, otherwise calls *api.getProfile()*. It listens for the custom *allminds:session-expired* event and redirects to /login?error=session_expired. It renders the sidebar (collapsible, remembered in localStorage) and mounts **AgentChat**.
- **components/AgentChat.tsx** - the floating 'Thandi' assistant. It sends free text with only *language_pref*, so the **supervisor LLM decides which agent answers** (no tool_hint).
- **components/SettingsPanel.tsx** - edits profile (PUT /profile), theme, language, and connects/disconnects Google Calendar (GET /calendar/connect?format=json, DELETE /calendar/disconnect).

- **LearnerCard, LearnerProgressModal, SensoryToolkitModal** - learner widgets that read/write progress records and sensory logs.
- **AuthShell** - the shared visual frame for login, signup and password pages.

2.2 Feature pages and the goal each one serves

Page	What the teacher does	Data / AI used
/dashboard	Overview: learner count, saved plans, parent letters, recent activity, to-do tasks (create + cycle status).	getLearners, getLessonPlans, getParentCommunications, getActivities, getTasks, createTask, updateTask
tools/assessment-planner	Generates lesson plan / worksheet / test / homework for grade+subject+topic, aligned to DCAPS; shows YouTube videos and AI images; save, reopen, delete, export PDF.	chatWithAgent(tool_hint='lesson', material_type, grade, subject, topic, support_need, difficulty); lesson-plans CRUD
tools/visual-support	Describe a topic/routine and get 2 visual supports, an interactive idea, a quick-make tip, AI-generated cards and videos.	chatWithAgent(tool_hint='visual')
tools/learner-profiles	Upload a CSV class list (parsed in the browser), view learners, export Excel.	getLearners, bulkUploadLearners (POST /learners/bulk, max 1000 rows)
tools/schedule-tracking	Per-learner daily timeline (time + title + notes); optionally pushes each item to Google Calendar.	schedule-items CRUD, getCalendarConnectUrl, createCalendarEvent
learnerAnalytics	Pick a learner; local KPIs (engagement, task completion, attendance, support priority band) + 'Analyse with agent' narrative.	getProgress, getSensoryLogs, chatWithAgent(tool_hint='analytics', records=[...])
community/parent-communication	Choose learner + progress/concern, add notes, AI drafts the letter; edit, save, download PDF.	chatWithAgent(tool_hint='parent', communication_type), createParentCommunication
community/forum	Teacher-to-teacher posts and comments.	getForumPosts, createForumPost, getForumComments, createForumComment
professional-development	Directory of support NGOs, filterable.	getNGOs

Table 2. Feature pages

3. How the frontend talks to everything else

3.1 lib/api.ts - the single integration point

Every network call goes through one function, *request(endpoint, options)*:

```
const API_BASE = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:5000/api';
headers: { 'Content-Type': 'application/json',       // skipped for FormData
           Authorization: `Bearer ${localStorage.allminds_token}` }
if (res.status === 401) { clear token+user; dispatch 'allminds:session-expired' }
throw new ApiError(message, code, status)   // code = body.error, e.g. 'email_not_verified'
```

The one exception is **chatWithAgent()**: it calls the **same-origin Next.js route** */api/agent/chat*, not Express directly. That route (app/api/agent/chat/route.ts) runs on the Next server, forwards the body and Authorization header to *${BACKEND_API_URL}/agent/chat* with a 185 s timeout, and normalises errors to *{error}*. Benefits: no CORS preflight for the long AI call, the agent URL is never exposed to the browser, and timeouts are controlled server-side.

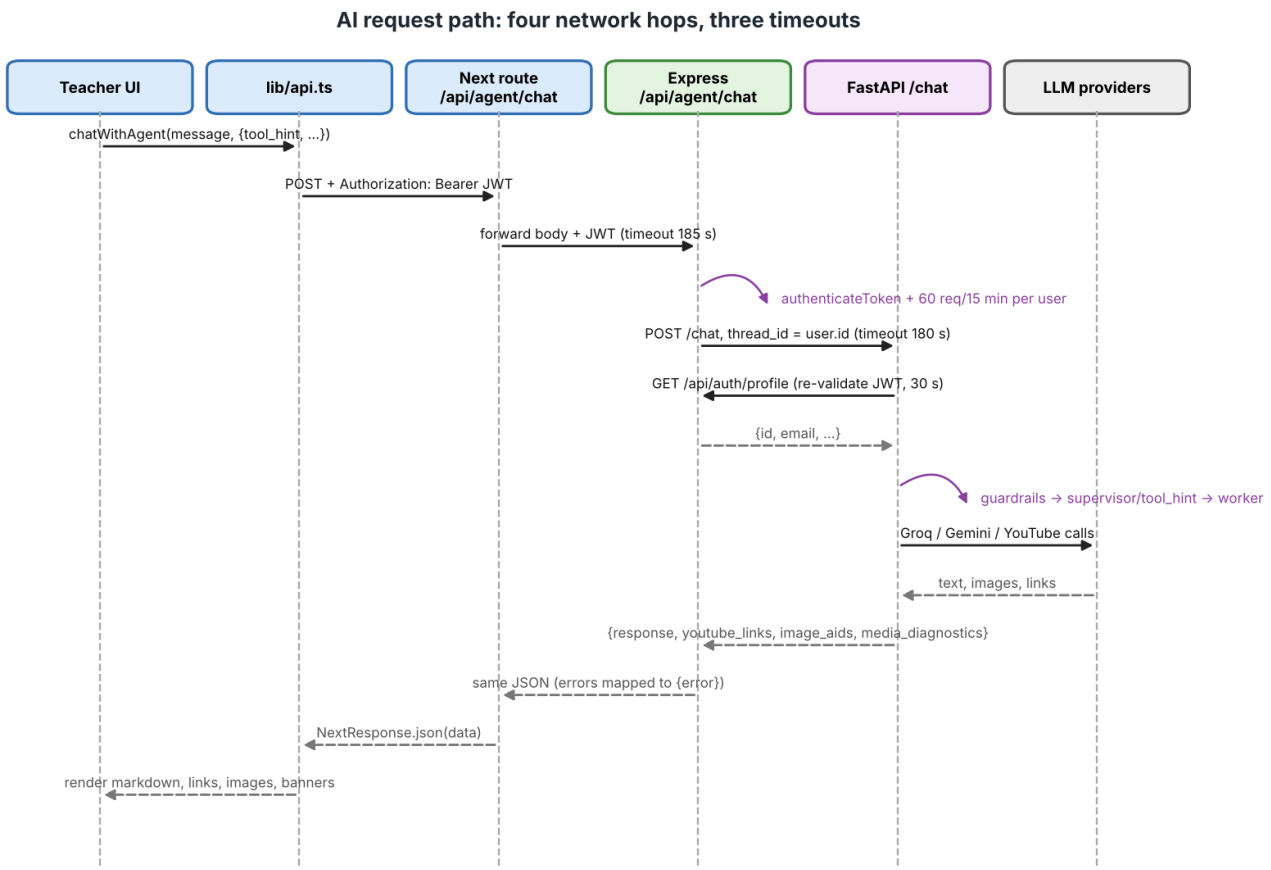


Figure 3. Sequence of one AI request. Note the agent service calls back into Express to verify the JWT, so the agent never needs the JWT secret.

Correction to system-design.json: that file says Next.js proxies directly to FastAPI. In the current code the path is Next.js -> Express /api/agent/chat -> FastAPI /chat. Express adds JWT verification and a per-user AI rate limit (60 requests / 15 min) before the agents are reached.

3.2 Complete API catalogue used by the frontend

lib/api.ts method	HTTP call (prefix /api)	Auth	Used by
login	POST /auth/login	-	login
register	POST /auth/register	-	signup
getOAuthUrl(provider)	browser redirect GET /auth/{google|microsoft|apple}	-	login, signup
exchangeOAuthCode	POST /auth/oauth/exchange {code}	-	oauth-callback
forgotPassword / resetPassword	POST /auth/forgot-password, /auth/reset-password	-	forgot/reset pages
verifyEmail / resendVerification	POST /auth/verify-email, /auth/resend-verification	-	verify-email, login, signup
getProfile / updateProfile	GET / PUT /profile	JWT	DashboardLayout, Settings, tools
getCalendarStatus / ConnectUrl / disconnect	GET /calendar/status, GET /calendar/connect?format=json, DELETE /calendar/disconnect	JWT	SettingsPanel, schedule-tracking
createCalendarEvent	POST /calendar/events	JWT	schedule-tracking
getLearners / createLearner / bulkUploadLearners	GET/POST /learners, POST /learners/bulk	JWT	dashboard, profiles, analytics, letters
get/createSensoryLog	GET/POST /learners/:id/sensory-logs	JWT	SensoryToolkitModal, analytics
get/createProgress	GET/POST /learners/:id/progress	JWT	LearnerProgressModal, analytics

lib/api.ts method	HTTP call (prefix /api)	Auth	Used by
forum methods	GET/POST /forum, GET/POST /forum/:id/comments	GET public, POST JWT	forum
parent communications	GET /parent-communications[?learnerId], POST	JWT	letters, dashboard
activities / lesson plans / tasks	GET /activities/activities, CRUD /activities/lesson-plans, /activities/tasks	JWT	dashboard, planner
schedule items	GET ?learnerId, POST, PATCH, DELETE /activities/schedule-items	JWT	schedule-tracking
getNGOs / strategies	GET /ngos, GET/POST /strategies, POST /strategies/:id/vote	mixed	professional-development
chatWithAgent	POST /api/agent/chat (Next route) -> Express -> FastAPI	JWT	all AI features

Table 3. Every call the frontend makes

3.3 Third-party APIs and how each serves the goal

External API	Called from	Purpose in AllMinds
Google OAuth 2.0 / Microsoft identity / Sign in with Apple	Express (Passport)	One-click teacher sign-in; Apple ID tokens verified against Apple JWKS (cached 1 h).
Google OAuth (calendar scope) + Calendar API v3 + oauth2 userinfo	Express calendarController; Python scheduling agent	Connect a teacher's calendar, push schedule items as events, let Thandi list/create events.
Groq API - llama-3.3-70b-versatile	Agents: supervisor	Ultra-fast intent classification (10 output tokens, temperature 0).
Google Gemini API - 2.5 Flash / 2.5 Pro / 2.5 Flash Image	Agents: all workers	Text generation, translation and native image generation of classroom visuals.
YouTube Data API v3 (search, safeSearch=strict)	Agents: visual_media.py	3 curated, embeddable videos per lesson/visual request (fallback: search-page links).
HuggingFace sentence-transformers all-MiniLM-L6-v2 + ChromaDB	Agents: RAG	Embeds DCAPS.pdf locally for curriculum-grounded answers.
SMTP via Nodemailer	Express	Password-reset / verification e-mails (dev mode returns the link when SMTP is not configured).

Table 4. External services

4. Key user journeys on the frontend

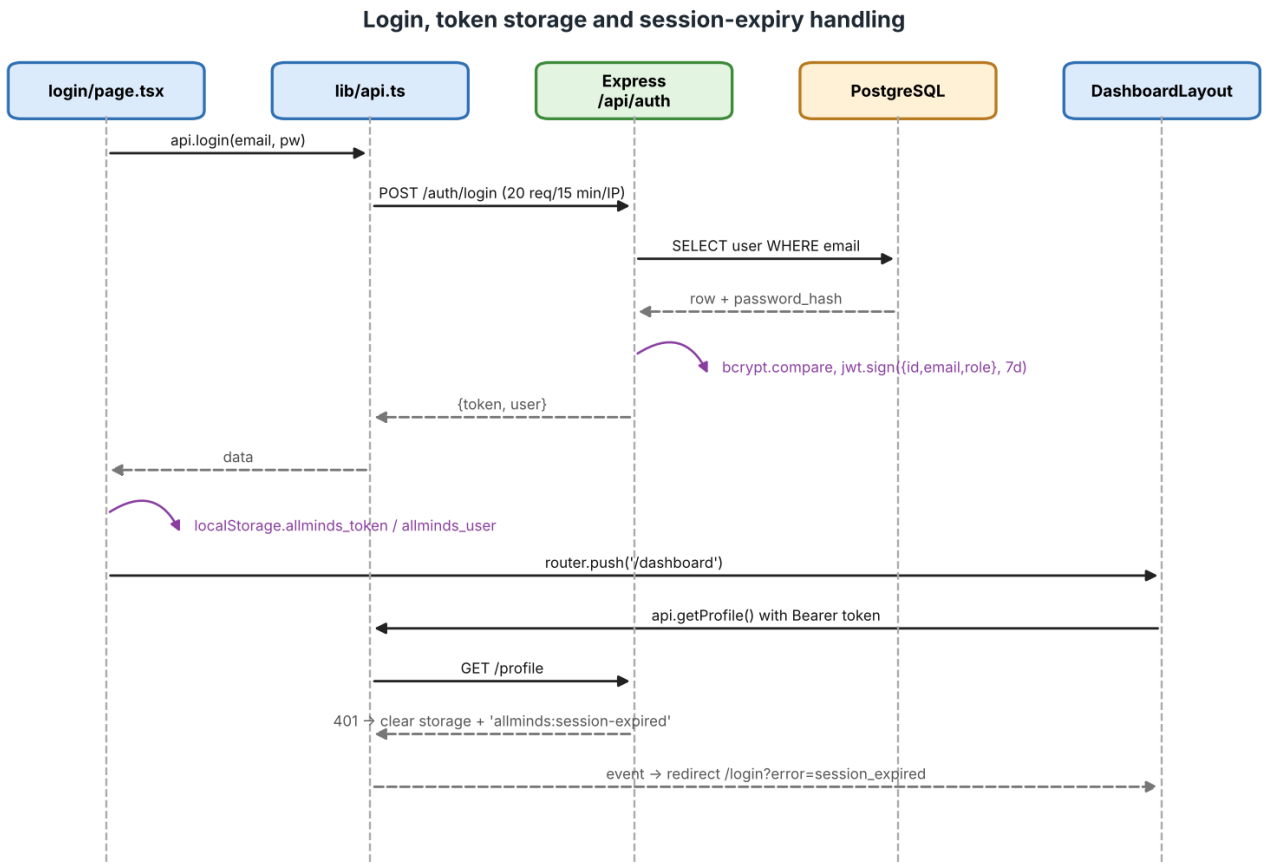


Figure 4. Email/password login. OAuth uses the same storage step after /oauth-callback exchanges a one-time code for the JWT.

4.1 OAuth (Google / Microsoft / Apple)

The login button sets *window.location* to */api/auth/google*. After the provider redirects back to Express, Express does **not** put the JWT in the URL; it stores a SHA-256 hash of a random 32-byte code (valid 5 minutes) and redirects to */oauth-callback?code=...*. The page calls *api.exchangeOAuthCode(code)*; the code is deleted atomically on use and a JWT is returned and stored like a normal login. This keeps tokens out of browser history and server logs.

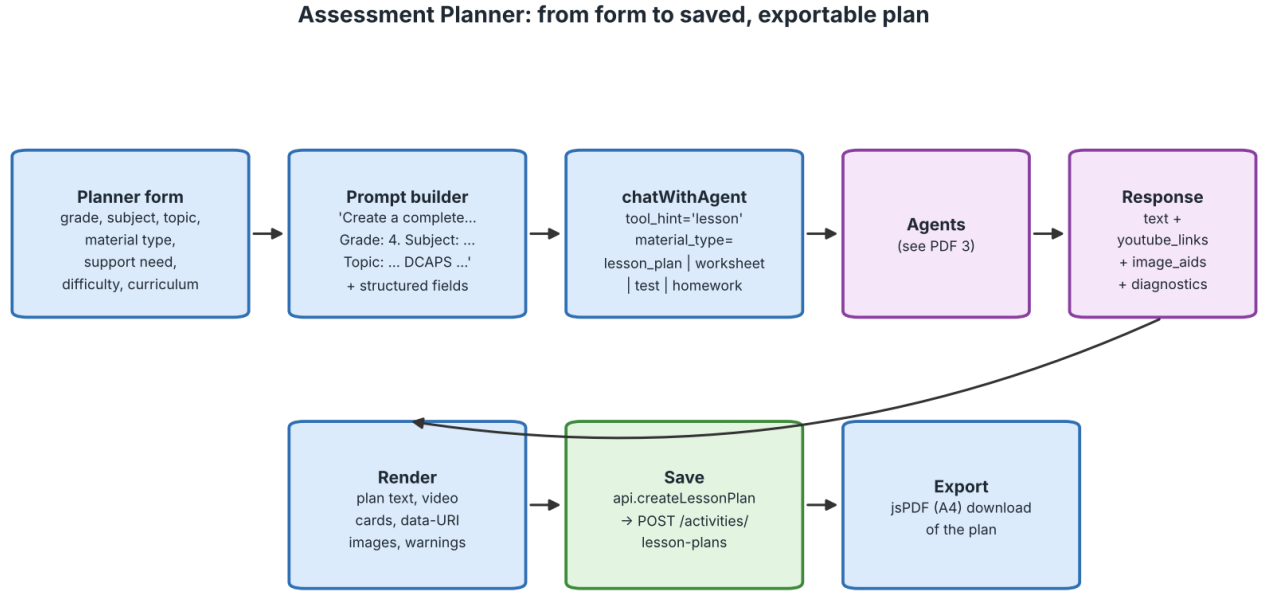


Figure 5. The Assessment Planner, the flagship tool, end to end.

4.2 Analytics page - local maths plus AI narrative

learnerAnalytics computes KPIs in the browser (engagement average, task completion, attendance rate, support-priority score banded High >= 70, Moderate >= 40, otherwise Low). For the AI narrative it maps each progress row into the agent's record format: *mark = engagementLevel x 20* (1-5 scale -> 0-100), behaviour = {attention, participation, homework_completion, emotional_regulation=3, attendance}, and sends *tool_hint='analytics'*.

4.3 Multilingual experience

The UI language and the AI response language are linked by one value - *allminds_language* (en, zu, xh, af, nso, tn, st, ss, ve, ts, nr). Pages pass it as *language_pref*; each agent appends a language instruction to its system prompt, and analytics translates its deterministic summary with Gemini.

4.4 Resilience in the UI

- Every AI response may include *media_diagnostics* - human-readable banners (e.g. 'Tip: add a YOUTUBE_API_KEY', 'Gemini quota exceeded'). The UI shows them without failing the text result.
- Errors from any hop are normalised to *{error}*, so pages show one consistent message.
- Long AI calls show loading states; timeouts surface as 'The AI request timed out. Please try again.'

5. Hackathon final - likely questions and model answers

Q1. Walk us through what happens when a teacher clicks 'Generate' in the Assessment Planner.

The page builds a natural-language prompt plus structured fields and calls api.chatWithAgent with tool_hint='lesson'. The browser posts to the same-origin Next route /api/agent/chat, which forwards to Express /api/agent/chat; Express verifies the JWT, applies a 60-per-15-minute per-user limit, overrides thread_id with the user id and posts to FastAPI /chat. FastAPI validates the token by calling Express /auth/profile, runs guardrails, skips the supervisor because of the tool hint, optionally enriches with DCAPS RAG context, and runs the lesson agent (Gemini 2.5 Pro text, YouTube search and Gemini image generation in parallel). JSON flows back through each hop and the page renders text, videos, images and diagnostics; the teacher can save it (POST /activities/lesson-plans) or export with jsPDF.

Q2. Why route AI calls through a Next.js API route instead of calling the agent service from the browser?

It removes CORS concerns for long requests, hides internal service URLs, lets us set a server-side timeout (185 s) slightly longer than Express's (180 s) so the innermost error surfaces cleanly, and keeps a single auth path through Express where rate-limiting and JWT checks live.

Q3. Where is the JWT stored and what are the trade-offs?

In localStorage ('allminds_token'), sent as a Bearer header. It is simple and works across three services, but is readable by any injected script (XSS). Mitigations/next steps: strict CSP, httpOnly secure cookies with CSRF protection, and shorter-lived access tokens with refresh.

Q4. How does the frontend react when a token expires?

Any 401 in lib/api.ts clears token and user from storage and dispatches 'allminds:session-expired'; DashboardLayout listens and redirects to /login?error=session_expired.

Q5. What is the difference between the floating Thandi chat and the tool pages?

Thandi sends free text with no tool_hint, so the Groq supervisor classifies the intent into one of eight agents. Tool pages know their purpose and send a tool_hint, skipping classification - lower latency, deterministic routing and richer structured fields.

Q6. How is learner privacy handled in the UI?

Learners are stored as anonymous_id + JSON profile data; the UI shows a pseudonym/display name derived in lib/learners.ts. Free text sent to the agents is PII-scrubbed (SA ID numbers, e-mails, phone numbers) by guardrails before any LLM call.

Q7. Which external APIs does the product depend on, and what happens if one fails?

Groq (routing), Gemini (generation, images, translation), YouTube Data API, Google Calendar, OAuth providers, SMTP. Failures degrade gracefully: no Groq key -> keyword routing; no YouTube key -> search links; image failure -> text still returned with a diagnostic banner; missing OAuth config -> the button redirects with '_oauth_not_configured'.

Q8. How do you support 11 languages?

The UI uses a translation dictionary keyed by language code; the same code is sent as language_pref and each agent appends a language instruction (e.g. respond in isiZulu, keeping technical terms in English in brackets where needed).

Q9. How would you scale the frontend?

It is stateless; static assets via CDN (Netlify/Render), the proxy route is the only server work. For AI latency, add streaming responses (SSE) and optimistic UI; cache NGO/forum reads.

Q10. What would you improve next on the frontend?

Streaming AI output, httpOnly cookie auth, a typed API client (api.ts uses 'any' in places), offline support for low-connectivity schools, and component tests for the tool pages.

5.1 Whole-system and pitch questions

Q1. In one sentence, what problem does AllMinds solve?

Most South African classroom teachers have neurodivergent learners but no specialist support; AllMinds gives them DCAPS-aligned, neurodivergent-affirming lesson materials, visual supports, parent letters and progress insight in seconds, in their own language.

Q2. Why is this 'agentic' rather than a chatbot?

A supervisor plans who should act, specialist agents use tools (vector search over DCAPS, YouTube, image generation, Google Calendar, deterministic analytics) and results are structured artefacts the teacher saves, exports or schedules - not just chat text.

Q3. How do the three services work together to reach the goal?

Next.js collects structured context and renders results; Express owns identity, learner data and integrations and gatekeeps every AI call; FastAPI routes to the right specialist and model, grounds content in the curriculum and returns safe, multilingual output that flows back to be saved in Postgres.

Q4. What is your demo path?

Log in -> upload a CSV class list -> generate a Grade 3 maths worksheet with DCAPS alignment (videos and images appear) -> save and export PDF -> log progress for a learner -> run the analytics narrative -> draft a parent progress letter in isiZulu -> ask Thandi a behaviour question.

Q5. How do you measure impact?

Time saved per lesson plan, number of artefacts saved/exported, learner progress trends over time, teacher satisfaction and language uptake; next we would add an evaluation set for content quality.